Edges Are Not All Equal: DropConnect With Dynamic Dropping Rates for Neural Network Regularization

Author Name
Affiliation
email@example.com

Abstract

Current Dropout or DropConnect applies a universal dropping rate to omit the neurons or edges of a neural network during training. We propose to leverage the magnitude of the gradients to assign different dropping rates to each edge. We conducted thorough experiments on a synthetic dataset and open datasets. The results show that this simple mechanism outperforms Dropout, DropConnect, and other variations.

1 Introduction

A deep neural network has a huge hypothesis space and learns to fit the relationship between the input features and the output targets among these hypotheses. However, a giant model tends to overfit the training data, especially when the number of training instances is insufficient. As a result, numerous techniques were proposed to address the issue of overfitting, e.g., introducing L1 or L2 penalties on the weights [Hastie *et al.*, 2009], applying early stopping [Goodfellow *et al.*, 2016], augmenting data instances [Krizhevsky *et al.*, 2017], and many more.

Among the various regularization approaches, Dropout [Hinton *et al.*, 2012; Srivastava *et al.*, 2014] and DropConnect [Wan *et al.*, 2013] are two popular techniques used by deep learning models. Dropout introduces a probability p such that each neuron has the probability p of being zeroed during training. Instead of dropping neurons, DropConnect randomly eliminates an edge with the probability p, so $(1 - p)$ portion of the inputs of a neuron are preserved. DropConnect can be regarded as a generalization of Dropout because disregarding one neuron (as Dropout does) corresponds to removing all the incoming and outgoing edges of this neuron (which can be achieved by DropConnect).

Dropout and DropConnect apply a universal dropping rate on the neurons or edges. However, a universal dropping rate may not be the best setting. Instead, a model should rely on more information to determine the dropping rates of the edges or the neurons and assign different dropping rates for different edges or different neurons.

We propose a new methodology DynamicDropConnect (DDC). DDC assigns the dropping probability of an edge based on the magnitude of the gradient of the edge. Given a neural network in which $e_{i,j}^{(l)}$ representing the edge connecting the neuron i at layer $l-1$ to the neuron j at layer l and $w_{i,j}^{(l)}$ representing the weight (i.e., parameter) of the edge, we hope to assign a higher dropping rate to the edges with smaller $\left|\partial L \,/\, \partial w_{i,j}^{(l)}\right|$ (L is the loss). The intuition is that if changing the value of a parameter only slightly influences the result, occasionally omitting this parameter is probably an acceptable choice.

We show a preliminary study to test this conjecture by comparing the following regularization methods and a non-regularized method on the linear regression model. First, training the model without regularization (denoted by "No regularization" in Figure 1); second, training the same model with DDC; and lastly, training the same model using some variants of DDC ("Drop Big Gradient", "Drop Small Parameter", and "Drop Big Parameter"). We show the values of the parameters (w_1 and w_2) and the losses at different epochs. We randomly initialize two sets of values for w_1 and w_2, as shown by the yellow crosses in the first row and the second row in Figure 1, respectively. As seen, training a linear regressor by DDC converges faster than the non-regularization method and the variants of DDC. The result motivates us to conduct more experiments to test the performance of DDC using more complex networks on open datasets.

The rest of the paper is organized as follows. First, in Section 2, we introduce the background knowledge and the details of the proposed DDC method. Then, section 3 shows the comparisons of the DDC approach with baseline methods (non-regularized method and variants of Dropout) using different network structures on a synthetic dataset and multiple open datasets. Next, we review the related studies in Section 4. Finally, we conclude our work and discuss future research directions in Section 5.

2 Methodology

We introduce the proposed DDC in this section.

2.1 Preliminary: DropConnect

Let $l \in \{1, \ldots, L\}$ be the L hidden layers in a neural network, and $\boldsymbol{y}^{(l)}$ be the output of layer l (and therefore also be the

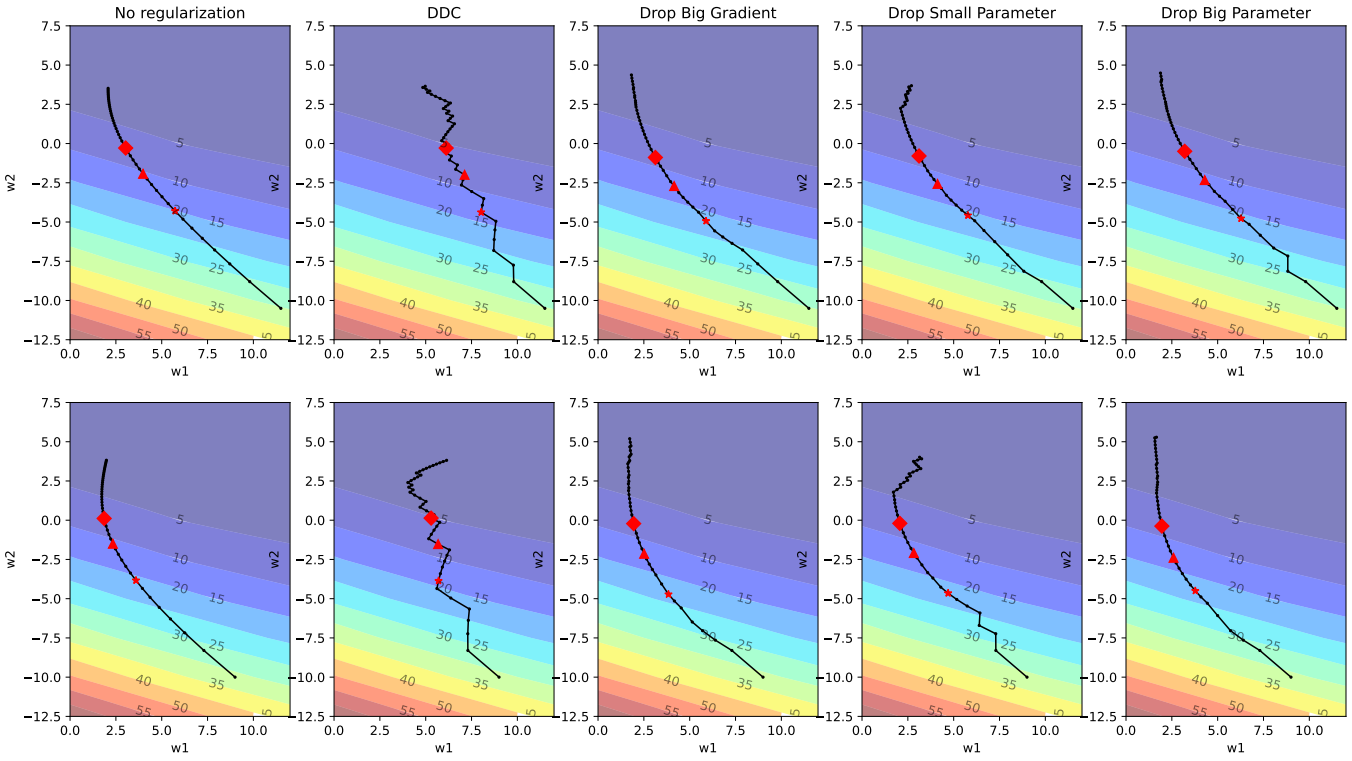


Figure 1: The contour plot of the parameter updating process. The two rows represent two different sets of initial values for w_1 and w_2 and their updating process. The yellow crosses denote the initial values. The red stars, triangles, and diamonds represent the values of w_1 and w_2 after training for 7 epochs, 13 epochs, and 19 epochs.

input of layer $l+1$), each layer of a neural network transforms $\boldsymbol{y}^{(l-1)}$ to $\boldsymbol{y}^{(l)}$ by the following Equation.

$$\boldsymbol{y}^{(l)} = f_l\left(\boldsymbol{z}^{(l)}\right) = f_l\left(\boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)}\right), \tag{1}$$

where $\boldsymbol{W}^{(l)}$ is the parameter matrix, and $f_l(\cdot)$ is the activation function for layer l.

The DropConnect mechanism assigns a universal dropping rate p such that each edge has a probability p to turn off during training. Therefore, a neural network transforms $\boldsymbol{y}^{(l-1)}$ to $\boldsymbol{y}^{(l)}$ (and continues proceeding to the following layers) by the equation below during training.

$$\boldsymbol{y}^{(l)} = f_l\left(\boldsymbol{z}^{(l)}\right) = f_l\left(\left((1-\boldsymbol{M}^{(l)})\odot\boldsymbol{W}^{(l)}\right)\boldsymbol{y}^{(l-1)}\right), \tag{2}$$

where $\boldsymbol{M}^{(l)} = [m_{i,j}]$ is a $(0,1)$-matrix in which each entry $m_{i,j}$ is sampled from a Bernoulli distribution with parameter p, and $\odot$ performs element-wise product. In other words, $\boldsymbol{M}^{(l)}$ is a mask matrix determines which parameters in $\boldsymbol{W}^{(l)}$ to omit.

2.2 Mask Generation for DDC

Our proposed DDC generalizes DropConnect by allowing different dropping probabilities for different edges. Eventually, we want a customized mask matrix $\widetilde{\boldsymbol{M}}^{(l)} = \left[\widetilde{m}_{i,j}^{(l)}\right]$ for each $\boldsymbol{W}^{(l)}$ to filter the parameters in $\boldsymbol{W}^{(l)}$, and each $\widetilde{m}_{i,j}^{(l)}$ is sampled from a Bernoulli distribution with distinct parameter $p_{i,j}^{(l)}$.

We show an pseudocode to generate $\widetilde{\boldsymbol{M}}^{(l)}$ in Algorithm 1; the following paragraphs explain the process.

Given an edge $e_{i,j}^{(l)}$ connecting the neuron i at the layer $l-1$ and the neuron j at the layer l, we use $w_{i,j}^{(l)}$ to denote the edge weight (i.e., parameter) of $e_{i,j}^{(l)}$ and $g_{i,j}^{(l)}$ to symbol the gradient (i.e., $g_{i,j}^{(l)} = \partial L \,/\, \partial w_{i,j}^{(l)}$, L is loss). Since we primarily care about the magnitude, not the sign, of a gradient, we compute the absolute value of $g_{i,j}^{(l)}$ and denote the result as $v_{i,j}^{(l)}$ (line 1 to line 5 in Algorithm 1).

Next, we compute the mean and standard deviation of the $v_{i,j}^{(l)}$-s for layer l (lines 6 and 7 in Algorithm 1) and standardize $v_{i,j}^{(l)}$ by z-score (as shown in line 10). As a result, the scale of the standardized scores $z_{i,j}^{(l)}$-s from different layers are similar.

We then compute $q_{i,j}^{(l)}$ for each edge $e_{i,j}^{(l)}$ based on the normalized magnitude of the gradient $z_{i,j}^{(l)}$. We call $q_{i,j}^{(l)}$ the candidate dropping probability because the value of $q_{i,j}^{(l)}$ is positively correlated with the dropping probability of the edge $e_{i,j}^{(l)}$. Eventually, we define the candidate dropping probability based on Equation 3, so the model tends to keep the edge with a larger magnitude of the gradient.

Algorithm 1: Generating a mask for $\boldsymbol{W}^{(l)}$

Input : weights $\boldsymbol{W}^{(l)} = \left[w_{i,j}^{(l)}\right] \in \mathcal{R}^{n_1 \times n_2}$,
gradients $\boldsymbol{G}^{(l)} = \left[g_{i,j}^{(l)}\right] \in \mathcal{R}^{n_1 \times n_2}$,
hyperparameters p, p_g, and τ

Output: mask $\widetilde{\boldsymbol{M}}^{(l)} = \left[\widetilde{m}_{i,j}^{(l)}\right] \in \mathcal{R}^{n_1 \times n_2}$

1 **for** $i \leftarrow 1$ **to** n_1 **do**
2 **for** $j \leftarrow 1$ **to** n_2 **do**
3 $v_{i,j}^{(l)} \leftarrow \left|g_{i,j}^{(l)}\right|$;
4 **end**
5 **end**
6 $\mu^{(l)} \leftarrow \dfrac{\sum_{i=1}^{n_1}\sum_{j=1}^{n_2} v_{i,j}^{(l)}}{n_1 \times n_2}$;
7 $\sigma^{(l)} \leftarrow \dfrac{\sum_{i=1}^{n_1}\sum_{j=1}^{n_2} (v_{i,j}^{(l)} - \mu^{(l)})^2}{n_1 \times n_2}$;
8 **for** $i \leftarrow 1$ **to** n_1 **do**
9 **for** $j \leftarrow 1$ **to** n_2 **do**
10 $z_{i,j}^{(l)} \leftarrow \dfrac{v_{i,j}^{(l)} - \mu^{(l)}}{\sigma^{(l)}}$;
11 Update $q_{i,j}^{(l)}$ by Equation 3;
12 Update $p_{i,j}^{(l)}$ by Equation 4;
13 $\widetilde{m}_{i,j}^{(l)} = \text{Bernoulli}(p_{i,j}^{(l)})$;
14 **end**
15 **end**

$$q_{i,j}^{(l)} \leftarrow \begin{cases} 1 - \sigma\left(z_{i,j}^{(l)}\right) & \text{if } 1 - \sigma\left(z_{i,j}^{(l)}\right) \geq \tau \\ 0 & \text{otherwise,} \end{cases} \tag{3}$$

where $\sigma()$ is the sigmoid function, and τ is a threshold hyper-parameter. Since $\sigma()$ is a monotonically increasing function, and the output is always between 0 and 1, a larger $z_{i,j}^{(l)}$ results in a smaller candidate dropping probability. We set $\tau = 0.5$ here below. As a result, on average half of the $q_{i,j}^{(l)}$-s are zero because the expected value of $\sigma\left(z_{i,j}^{(l)}\right)$ is 0.5.

We compute the final dropping rate $p_{i,j}^{(l)}$ for the edge $e_{i,j}^{(l)}$ by combining the candidate dropping probability $q_{i,j}^{(l)}$ with the base dropping probability p, as shown below.

$$p_{i,j}^{(l)} \leftarrow \begin{cases} p + p_g \times q_{i,j}^{(l)} & \text{if } p + p_g \times q_{i,j}^{(l)} \leq 1 \\ 1 & \text{otherwise,} \end{cases} \tag{4}$$

where both p (we name "base dropping rate") and p_g (we name "gradient unit dropping rate") are hyperparameters. We limit the value of $p_{i,j}^{(l)}$ to be no greater than 1 to ensure $p_{i,j}^{(l)}$ as a proper probability value.

Eventually, for an edge $e_{i,j}^{(l)}$, we sample the corresponding mask value $\widetilde{m}_{i,j}^{(l)}$ by a Bernoulli distribution with parameter $p_{i,j}^{(l)}$ (line 13).

Algorithm 2: One-layer forward re-calibration during training

Input : weights $\boldsymbol{W}^{(l)} = \left[w_{i,j}^{(l)}\right] \in \mathcal{R}^{n_1 \times n_2}$,
gradients $\boldsymbol{G}^{(l)} = \left[g_{i,j}^{(l)}\right] \in \mathcal{R}^{n_1 \times n_2}$,
outputs from previous layer $\boldsymbol{y}^{(l-1)} \in \mathcal{R}^{n_2 \times 1}$

Output: $\boldsymbol{y}^{(l)} \in \mathcal{R}^{n_1 \times 1}$

1 Compute $\widetilde{\boldsymbol{M}}^{(l)}$ by Algorithm 1 using $\boldsymbol{W}^{(l)}$ and $\boldsymbol{G}^{(l)}$;
2 $\widetilde{\boldsymbol{W}}^{(l)} \leftarrow \left(1 - \widetilde{\boldsymbol{M}}^{(l)}\right) \odot \boldsymbol{W}^{(l)}$;
3 Compute $r^{(l)}$ by Equation 5;
4 Compute $\boldsymbol{y}^{(l)}$ by Equation 6;

2.3 Training Re-calibration and Testing

This section presents the process to forward from $\boldsymbol{y}^{(l-1)}$ (the output of the previous layer) to $\boldsymbol{y}^{(l)}$ during training. We show the pseudocode in Algorithm 2.

As introduced in Section 2.2, $\widetilde{m}_{i,j}^{(l)}$ serves as a mask for the parameter $w_{i,j}^{(l)}$, i.e., if $\widetilde{m}_{i,j}^{(l)} = 1$, the corresponding $w_{i,j}^{(l)}$ should be omitted. Therefore, the effective parameters at layer l go from $\boldsymbol{W}^{(l)}$ (training without regularization) to $(1 - \widetilde{\boldsymbol{M}}^{(l)}) \odot \boldsymbol{W}^{(l)}$ (training using DDC regularization with mask $\widetilde{\boldsymbol{M}}^{(l)}$) .

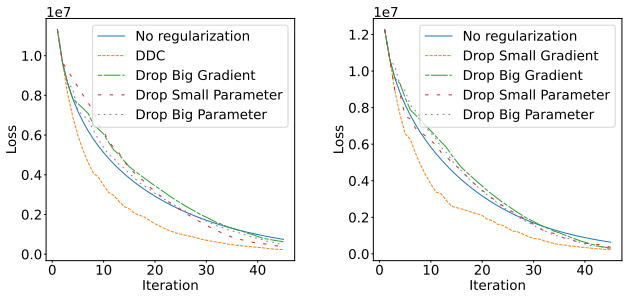
The dropping mechanism is only applied during training but not during prediction. However, if we set the value of $\boldsymbol{y}^{(l)}$ as $f_l\left((1 - \widetilde{\boldsymbol{M}}^{(l)}) \odot \boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)}\right)$ during training (i.e., apply DDC masking) but predict $\boldsymbol{y}^{(l)}$ as $f_l\left(\boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)}\right)$ after training (i.e., without DDC masking), apparently the learned $\boldsymbol{W}^{(l)}$ from training cannot be directly used during testing. To fix the inconsistency, we re-calibrate $(1 - \widetilde{\boldsymbol{M}}^{(l)}) \odot \boldsymbol{W}^{(l)}$ during training such that the learned $\boldsymbol{W}^{(l)}$ can directly be used during prediction by $f_l\left(\boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)}\right)$. We introduce the re-calibrating mechanism below.

Since $\widetilde{\boldsymbol{M}}^{(l)}$ is a $(0,1)$-matrix, $\sum_{i=1}^{n_1}\sum_{j=1}^{n_2} \widetilde{m}_{i,j}^{(l)}$ is the number of 1-s in the matrix $\widetilde{\boldsymbol{M}}^{(l)}$. Therefore, the dropping rate (i.e., percentage of the entries in $\boldsymbol{W}^{(l)}$ being masked by the matrix $\widetilde{\boldsymbol{M}}^{(l)}$) can be calculated by Equation 5.

$$r^{(l)} = \frac{\sum_{i=1}^{n_1}\sum_{j=1}^{n_2} \widetilde{m}_{i,j}^{(l)}}{n_1 \times n_2}, \tag{5}$$

where n_1 and n_2 are the row count and the column count of $\widetilde{\boldsymbol{M}}^{(l)}$, respectively.

We re-calibrate the output of layer l during training such that the learned weights $\boldsymbol{W}$ can be used directly during prediction. The re-calibration is achieved by dividing the output by the keep rate (i.e., 1 subtract the parameter dropping rate), as shown by the equation below.



(a) $w_1 = 11.5, w_2 = -10$ (b) $w_1 = 9, w_2 = -10.5$

Figure 2: Loss vs. epochs of different models.

$$\boldsymbol{y}^{(l)} \leftarrow f_l\left(\widetilde{\boldsymbol{W}}^{(l)}\boldsymbol{y}^{(l-1)} \times \frac{1}{1-r^{(l)}}\right). \tag{6}$$

The mask matrix $\widetilde{\boldsymbol{M}}^{(l)}$ and the masked parameter matrix $\widetilde{\boldsymbol{W}}^{(l)}$ are only used during training. At the prediction phase, we use only the unmasked parameter matrix $\boldsymbol{W}^{(l)}$ to convert $\boldsymbol{y}^{(l-1)}$ into $\boldsymbol{y}^{(l)}$, as shown below.

$$\boldsymbol{y}^{(l)} \leftarrow f_l\left(\boldsymbol{W}^{(l)}\boldsymbol{y}^{(l-1)}\right). \tag{7}$$

3 Experiments

This section introduces the experiment settings and results. All the models are implemented by PyTorch and trained on the NVIDIA GTX 3090. We conducted experiments based on one synthetic dataset and four open datasets: MNIST [LeCun *et al.*, 1998], CIFAR-10 [Krizhevsky *et al.*, 2009], CIFAR-100 [Krizhevsky *et al.*, 2009], and NORB [LeCun *et al.*, 2004]. We split the labeled instances for each open dataset into the training set, the validation set, and the test set. The detailed settings, such as parameter initialization, learning rate, and batch size, are included in the experimental code, which will be released after acceptance.

3.1 Experiments on the Synthetic Dataset

To analyze and compare the learning process of the non-regularization method, DDC regularization, and some variants of DDC, we generated a synthetic dataset as follows. For each instance i, we sampled two independent features $x_1^{(i)}$ and $x_2^{(i)}$ from the standard normal distribution. We fixed the values of w_1 and w_2 and let the corresponding target $y^{(i)}$ be $w_1 x_1^{(i)} + w_2 x_2^{(i)} + \xi$, where ξ also follows a standard normal distribution. We only used two parameters (w_1 and w_2) because we want to visualize the updating process of their values

Figure 1 gives the comparison of training without regularization and with DDC regularization along with some DDC variants: "Drop Big Gradient" tends to drop the edges with larger gradients; "Drop Small Parameter" favors dropping edges with small weights; "Drop Big Parameter" tends to drop edges with large weights. The horizontal and vertical axes denote the values of the estimated w_1 and w_2 at different epochs, and the contour lines represent the loss. We highlight the initial values of w_1 and w_2 and their values at the 7th, 13th, and 19th epochs by yellow crosses, red stars, red triangles, and red diamonds, respectively. As shown, updating parameters by DDC regularization reaches small losses faster. We randomly assign the values to w_1 and w_2 as the initial value and repeat the process several times. We show only two cases in the first row and the second row of Figure 1, but other initialization yields similar results.

To give a quantitative comparison, we show the relationship between the epochs and losses of the compared methods in Figure 2. The results show that DDC gets the lowest loss when giving a fixed epoch count. The methodologies "Drop Small Parameter", "Drop Big Parameter", and "No regularization" are in the middle, and "Drop Big Gradient" apparently yields the largest loss. Again, we demonstrate the results using two different sets of initial values for w_1 and w_2, as shown by Figure 2a and Figure 2b, respectively, Other initial value sets give similar patterns.

3.2 Experiments on Open Datasets

The above section shows that DDC helps linear models learn faster. This section explores the capacity of DDC on more complicated deep learning models. We use four open datasets for experiments: MNIST, CIFAR-10, CIFAR-100, and NORB.

In addition to the non-regularization and the regularization strategies introduced in Section 3.1, we also included Dropout and DropConnect as the compared baselines in this section. For a fair comparison, we adjust the dropping rate $r^{(l)}$ in Algorithm 2 to be close to the dropping probability of Dropout and DropConnect for all the following experiments unless otherwise mentioned.

We tested the convolutional neural network used in the DropConnect paper [Wan *et al.*, 2013] (we call SimpleCNN below). In addition, we added two more complicated networks –AlexNet [Krizhevsky *et al.*, 2017] and VGG [Simonyan and Zisserman, 2014] – to compare the power of different regularization strategies.

MNIST

The first experimental dataset, MNIST, contains gray-scale images; each is with size 28×28. We re-scale the value of each pixel into $[0, 1]$.

Since the task is relatively simple, we tested only different regularization strategies on the SimpleCNN network, which is composed of three convolutional layers and two fully connected layers. The detailed network structure is illustrated in [Wan *et al.*, 2013]. We train the model for 100 epochs with a batch size of 128. We applied stochastic gradient descent (SGD) with a momentum of 0.9 and a learning rate of 0.001. We used the cosine annealing scheduler to set the learning rate [Loshchilov and Hutter, 2017]. We used the warm-up technique for 12 epochs [Loshchilov and Hutter, 2016]; other detailed settings are included in the experimental code, which will be released after acceptance.

The results are in Table 1. We report the average $\pm$ standard deviation obtained by repeating each experiment five

Table 1: The results of MNIST on the SimpleCNN network. We repeat each experiment 5 times and report accuracy $\pm$ standard deviation. Let the test accuracy of the "no regularization" method and the test accuracy of a regularization method m be $a \pm b$ and $c \pm d$, respectively. If $a + b < c - d$, we label the regularization method m with symbol ($\uparrow\uparrow$). If $a < c$ but $a + b \not< c - d$, we denote the regularization method m with the symbol ($\uparrow$). If $a - b > c + d$, we denote the regularization method m by ($\downarrow\downarrow$) symbol. Finally, if $a > c$ but $a - b \not> c + d$, we label the ($\downarrow$) symbol.

Method	Accuracy (%)
No regularization	99.08 ± 0.04
DDC	$\mathbf{99.25 \pm 0.01}(\uparrow\uparrow)$
Dropout	$99.03 \pm 0.08(\downarrow)$
DropConnect	$99.22 \pm 0.03(\uparrow\uparrow)$
Drop Small Parameter	$99.24 \pm 0.01(\uparrow\uparrow)$
Drop Big Parameter	$99.10 \pm 0.01(\uparrow)$
Drop Big Gradient	$99.12 \pm 0.01(\uparrow)$

Table 2: The results of CIFAR-10 for the SimpleCNN network, the symbols ($\uparrow\uparrow$), ($\uparrow$), ($\downarrow$), and ($\downarrow\downarrow$) are the same as in Table 1.

Method	Test Accuracy (%)
No regularization	77.17 ± 0.12
DDC	$80.12 \pm 0.04(\uparrow\uparrow)$
Dropout	$\mathbf{80.18 \pm 0.07}(\uparrow\uparrow)$
DropConnect	$80.01 \pm 0.03(\uparrow\uparrow)$
Drop Small Parameter	$79.15 \pm 0.07(\uparrow\uparrow)$
Drop Big Parameter	$79.64 \pm 0.04(\uparrow)$
Drop Big Gradient	$79.39 \pm 0.03(\uparrow)$

times. As demonstrated, the test accuracies mostly increase when applying the regularization strategies, and the proposed DDC has the best accuracy among all compared methods.

CIFAR-10

CIFAR-10 contains 32×32 color images. Thus, we changed the input dimension from (28, 28) (used in MNIST) to (32, 32, 3). We scaled the values of each input RGB channel to the $[0, 1]$ interval. Additionally, we added AlexNet and VGG network structures for experiments because CIFAR-10 is more complex than MNIST. The AlexNet comprises five convolutional layers and three fully connected layers [Krizhevsky *et al.*, 2017]. The VGG network contains ten convolutional layers and three fully connected layers [Simonyan and Zisserman, 2014].

Table 3: The results of CIFAR-10 on the AlexNet, the symbols ($\uparrow\uparrow$), ($\uparrow$), ($\downarrow$), and ($\downarrow\downarrow$) are the same as in Table 1.

Model	Test Accuracy (%)
No regularization	81.04 ± 0.04
DDC	$\mathbf{84.25 \pm 0.09}(\uparrow\uparrow)$
Dropout	$83.86 \pm 0.09(\uparrow\uparrow)$
DropConnect	$83.52 \pm 0.17(\uparrow\uparrow)$
Drop Small Parameter	$83.00 \pm 0.10(\uparrow\uparrow)$
Drop Big Parameter	$83.81 \pm 0.06(\uparrow)$
Drop Big Gradient	$83.19 \pm 0.16(\uparrow)$

Table 4: The results of CIFAR-10 on VGG, the symbols ($\uparrow\uparrow$), ($\uparrow$), ($\downarrow$), and ($\downarrow\downarrow$) are the same as in Table 1.

Model	Test Accuracy (%)
No regularization	90.46 ± 0.12
DDC	$\mathbf{90.94 \pm 0.11}(\uparrow\uparrow)$
Dropout	$90.66 \pm 0.07(\uparrow\uparrow)$
DropConnect	$90.68 \pm 0.08(\uparrow\uparrow)$
Drop Small Parameter	$89.88 \pm 0.01(\downarrow\downarrow)$
Drop Big Parameter	$90.45 \pm 0.02(\downarrow)$
Drop Big Gradient	$90.88 \pm 0.03(\uparrow\uparrow)$

Table 5: The results of CIFAR-100 on VGG, the symbols ($\uparrow\uparrow$), ($\uparrow$), ($\downarrow$), and ($\downarrow\downarrow$) are the same as in Table 1.

Model	Test Accuracy (%)
No regularization	71.38 ± 0.03
DDC	$\mathbf{72.09 \pm 0.04}(\uparrow\uparrow)$
Dropout	71.69 ± 0.06 ($\uparrow\uparrow$)
DropConnect	71.60 ± 0.21 ($\uparrow$)
Drop Small Parameter	71.71 ± 0.10 ($\uparrow\uparrow$)
Drop Big Parameter	71.82 ± 0.03 ($\uparrow\uparrow$)
Drop Big Gradient	71.13 ± 0.07 ($\downarrow\downarrow$)

Table 2 shows the results of applying various regularization techniques on SimpleCNN using the CIFAR-10 dataset. All the regularization methods perform better than the non-regularization strategy regarding their test accuracies. Dropout gives the best result in this setting, followed by our DDC strategy.

Table 3 and Table 4 show the accuracies of CIFAR-10 on AlexNet and VGG, respectively. The regularization methods mostly increase the test accuracies, and our DDC performs the best among all methods when using either the AlexNet or the VGG network.

CIFAR-100

We tested VGG on CIFAR-100 by changing the output softmax layer to 100 categories; all other settings are identical to the previous setup.

Table 5 shows the results of CIFAR-100 on the VGG network. The test accuracies of the regularization methods are mostly better than the non-regularization approach. Our DDC outperforms all the baseline methods.

NORB

Table 6 and Table 7 show the results of the NORB dataset on the AlexNet and the VGG network, respectively. It appears that not all regularization methods work well when applying AlexNet on NORB. However, all regularization methods outperform the non-regularization method when applying VGG on NORB. Our proposed DDC still performs best in all cases.

Table 6: The results of NORB on AlexNet, the symbols ($\Uparrow$), ($\uparrow$), ($\downarrow$), and ($\Downarrow$) are the same as in Table 1.

Model	Test Accuracy (%)
No regularization	92.08 ± 0.11
DDC	$\mathbf{92.93 \pm 0.09}(\Uparrow)$
Dropout	92.04 ± 0.07 ($\downarrow$)
DropConnect	$92.09 \pm 0.14(\uparrow)$
Drop Small Parameter	92.42 ± 0.10 ($\Uparrow$)
Drop Big Parameter	$92.02 \pm 0.04(\downarrow)$
Drop Big Gradient	91.85 ± 0.13 ($\downarrow$)

Table 7: The results of NORB on VGG, the symbols ($\Uparrow$), ($\uparrow$), ($\downarrow$), and ($\Downarrow$) are the same as in Table 1.

Model	Test Accuracy (%)
No regularization	93.36 ± 0.03
DDC	$\mathbf{94.20 \pm 0.16}(\Uparrow)$
Dropout	$93.79 \pm 0.25(\Uparrow)$
DropConnect	$93.94 \pm 0.19(\Uparrow)$
Drop Small Parameter	$94.02 \pm 0.05(\Uparrow)$
Drop Big Parameter	$93.82 \pm 0.15(\Uparrow)$
Drop Big Gradient	$94.17 \pm 0.07(\Uparrow)$

3.3 A Comparison on the Distribution of the Learned Parameter Values

This section compares the distribution of the learned parameters of the non-regularized method, DDC, Dropout, and DropConnect. We use SimpleCNN as the experimental network structure in this section. In addition, we use MNIST and CIFAR-10 as the experimental datasets.

Figure 3 shows the result of applying SimpleCNN on MNIST using the non-regularized method, Dropout, DropConnect, and DDC. As expected, the final parameter values generated by all regularization methods are closer to zero when compared with the non-regularized approach. If we define the close-to-zero numbers as those lying in the $[-0.1, 0.1]$ range, the non-regularized method outputs approximately 400 close-to-zero parameters. On the other hand, Dropout with dropping rates of 0.5 and 0.6 generate 550 and 450 close-to-zero parameters, respectively. In other words, a greater dropping rate for Dropout does not lead to a larger number of parameters whose values are close to zero. As for DropConnect and DDC, the number of close-to-zero parameters is more than 550.

We observe a similar trend on the CIFAR-10 dataset. As shown in Figure 4, all regularization methods generate more close-to-zero parameters. Particularly, non-regularized methods generate approximately 1500 close-to-zero parameters, Dropout yields 1700 close-to-zero parameters, and DropConnect and DDC generate roughly 2000 such parameters.

The experimental results are consistent on different datasets: Dropout has a stronger regularization strength than the non-regularization technique, and DropConnect and DDC have an even more considerable regularization strength than Dropout. DropConnect and DDC have stronger regularization strength than Dropout, likely because DropConnect and DDC omit the edges directly during training, whereas Dropout discards the neurons, not the edges. However, the fundamental reasons still require further study.

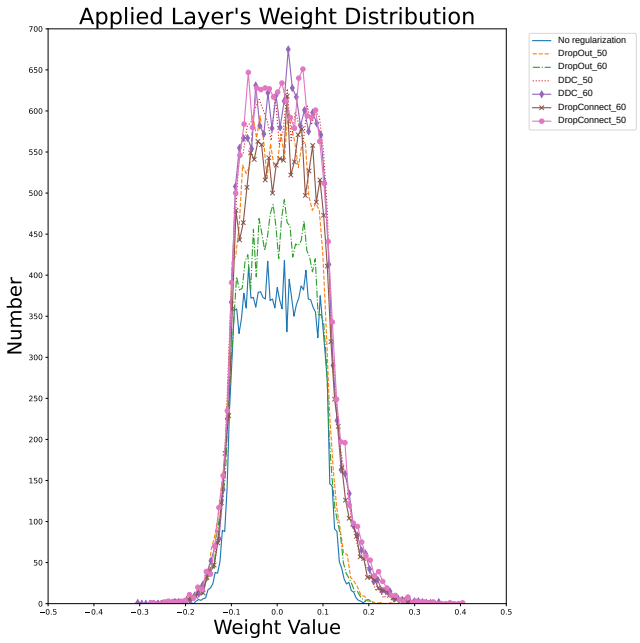


Figure 3: The distribution of the learned values of the parameters with different regularization methods on the MNIST dataset. DropConnect and our proposed DDC result in the strongest regularization effect.

4 Related Work

Overfitting refers to the scenario where a model performs well on the training data but is unsatisfactory on the unseen test data. To prevent overfitting, a common strategy is adding a penalty term to the objective function, such as the L1-norm, L2-norm, or some of their variants on the parameters [Hastie *et al.*, 2009]. As a result, these penalty terms encourage forming a simpler relationship between the features and the targets. Early stopping is another widely used method to prevent overfitting. Early stopping updates the parameters until the loss on the validation set appears to increase. Other popular methods include data augmentation [Krizhevsky *et al.*, 2017], model ensemble [Ke *et al.*, 2017], reducing the depth or width of a neural network, etc.

Dropout is a simple way for neural networks to prevent overfitting [Srivastava *et al.*, 2014]. Dropout randomly ignores part of the neurons during training. As a result, a neural network learns to fit the target without relying too much on a small portion of neurons. Additionally, randomly disregarding neurons during training functions as using multiple models and averaging the predictions. As a result, Dropout becomes an essential technique in training a deep neural network. In practice, the Inverted Dropout is commonly used to maintain the same expected value for the training and testing outputs.

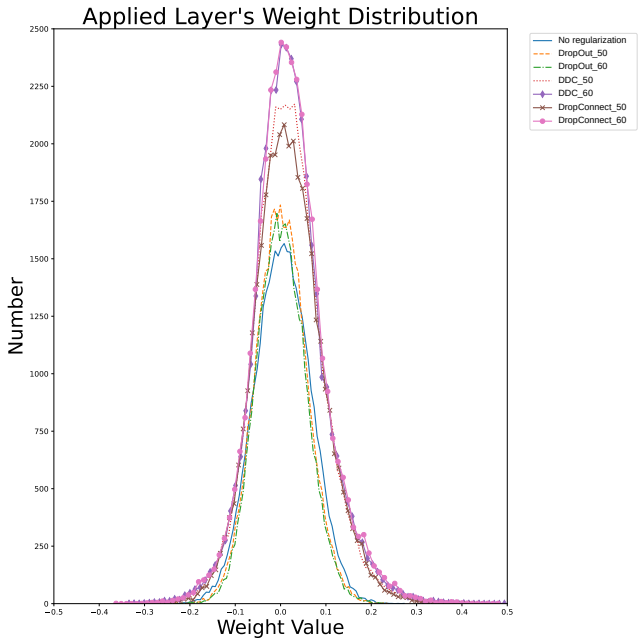


Figure 4: The distribution of the learned values of the parameters with different regularization methods on the CIFAR-10 dataset. DropConnect and our proposed DDC result in the most substantial regularization effect.

Instead of applying a random mask on the neurons, DropConnect randomly omits the edges during training. DropConnect can be regarded as an extension of Dropout because masking a neuron by Dropout is equivalent to omitting both the input edge and the output edge of the neuron by DropConnect. In practice, Dropout and DropConnect often yield models with similar accuracies.

Dropout can also be explained from the Bayesian perspective. Particularly, Dropout equivalents to an approximated Bayesian inference in deep Gaussian processes [Gal and Ghahramani, 2016]. Famous techniques in this line include Monte Carlo Dropout [Gal and Ghahramani, 2016], fast dropout [Wang and Manning, 2013], variational dropout [Kingma *et al.*, 2015], etc.

Instead of using a fixed dropping rate, some studies proposed adjusting the dropping rate during training. A classic example is standout [Ba and Frey, 2013], which learns the dropping rate based on the magnitude of the parameters. The standout is similar to our work because both alter the dropping rate on-the-fly during training. However, there are at least two manifest differences. First, we use the magnitude of the gradients, not the parameters, to decide the dropping rates. Second, our method involves no extra learning mechanism, whereas standout requires learning additional parameters to determine the dropping rate of the edges, which requires more training time.

A survey of Dropout and its variations can be found in [Labach *et al.*, 2019].

5 Conclusion

This paper introduces a simple and effective strategy to decide the random dropping rate of the edges of a neural network. This strategy is highly efficient because it involves no extra parameter learning. We conducted experiments to show the parameter learning process using a synthetic dataset and compared the proposed methodology with Dropout and its variants based on several open datasets. Experimental results show that the proposed method outperforms the compared baselines in nearly all cases.

Since the magnitude of the gradient is shown as a decent indicator of the dropping rate, we plan to use the gradients as the features and let the machines learn the relationship between the gradients and the proper dropping rates. This strategy may further boost prediction accuracy. However, such a strategy may require a longer training and prediction time.

We would also like to explore the theoretical analysis of our method. For example, the relationship between Dropout and Bayesian learning was studied in [Gal and Ghahramani, 2016]. Therefore, we can also explore the relationship between our approach and Bayesian learning.

References

[Ba and Frey, 2013] Jimmy Ba and Brendan Frey. Adaptive dropout for training deep neural networks. *Advances in neural information processing systems*, 26, 2013.

[Gal and Ghahramani, 2016] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.

[Goodfellow *et al.*, 2016] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[Hastie *et al.*, 2009] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer, 2009.

[Hinton *et al.*, 2012] Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.

[Ke *et al.*, 2017] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

[Kingma *et al.*, 2015] Durk P Kingma, Tim Salimans, and Max Welling. Variational dropout and the local reparameterization trick. *Advances in neural information processing systems*, 28, 2015.

[Krizhevsky *et al.*, 2009] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, University of Toronto, ON, Canada, 2009.

[Krizhevsky *et al.*, 2017] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017.

[Labach *et al.*, 2019] Alex Labach, Hojjat Salehinejad, and Shahrokh Valaee. Survey of dropout methods for deep neural networks. *arXiv preprint arXiv:1904.13310*, 2019.

[LeCun *et al.*, 1998] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[LeCun *et al.*, 2004] Yann LeCun, Fu Jie Huang, and Leon Bottou. Learning methods for generic object recognition with invariance to pose and lighting. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, volume 2, pages II–104. IEEE, 2004.

[Loshchilov and Hutter, 2016] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

[Loshchilov and Hutter, 2017] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[Simonyan and Zisserman, 2014] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[Srivastava *et al.*, 2014] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[Wan *et al.*, 2013] Li Wan, Matthew Zeiler, Sixin Zhang, Yann Le Cun, and Rob Fergus. Regularization of neural networks using dropconnect. In *International conference on machine learning*, pages 1058–1066. PMLR, 2013.

[Wang and Manning, 2013] Sida Wang and Christopher Manning. Fast dropout training. In *international conference on machine learning*, pages 118–126. PMLR, 2013.